Projet Synthèse: Mise en Place, Virtualisation et Monitoring d'un Écosystème d'IA

Cours : Mise en place d'un écosystème d'IA (AEC - Spécialisation technique en IA)
Enseignant : Dr. Komi Sodoké
Durée maximale : 10 heures
Plateforme d'exécution : Google Colab

1. Présentation générale et Objectifs

Ce projet d'intégration pratique a pour but de valider vos compétences acquises à travers les modules 1 à 6 du cours. En tant que futur technicien spécialisé, vous devez concevoir, configurer, virtualiser et mettre sous surveillance un mini-écosystème dédié à l'entraînement d'un modèle d'intelligence artificielle.
Le projet est conçu pour être entièrement réalisable dans l'environnement **Google Colab** pour la partie pratique, accompagné de réponses de réflexion technique.

Élément de Compétence Visé	Application Concrète dans le Projet
1. Cerner les besoins matériels	Choix de l'architecture de calcul et calcul des ratios CPU/GPU et lignes PCIe.
2. Mettre en place l'environnement matériel	Vérification des pilotes et de l'état du système d'exploitation via commandes Linux.
3. Mettre en place l'environnement	Configuration du framework

Élément de Compétence Visé	Application Concrète dans le Projet
de dév.	(PyTorch/TensorFlow) et exécution d'une tâche de calcul.
4. Utilisation de la virtualisation	Rédaction d'un fichier de conteneurisation Dockerfile optimisé pour l'IA.
5. Contrôler et entretenir l'infrastructure	Développement d'une boucle de monitoring (Watchdog) simulant des pannes matérielles.

2. Structure du Projet (4 Parties Pratiques)

Partie 1 : Dimensionnement et Choix Matériel (Théorique)

Votre startup doit entraîner un grand modèle de traitement de langage (LLM). On vous propose deux choix d'infrastructures cloud à louer :

- **Option A :** Serveur avec 1x GPU NVIDIA A100 (40 Go VRAM), CPU 8 cœurs, 16 lignes PCIe Gen 4.
- **Option B :** Serveur avec 4x GPU NVIDIA RTX 4090 (24 Go VRAM chacun), CPU 16 cœurs, 24 lignes PCIe Gen 4 partagées.

Travail demandé : En vous basant sur les notions d'optimisation de la chaîne de calcul (goulots d'étranglement PCIe, nombre de cœurs CPU par GPU pour alimenter les données, coûts), déterminez quelle option est la plus équilibrée pour éviter le bridage du matériel. Justifiez votre choix en quelques lignes.

Partie 2 : Audit du Système et Environnement de Calcul (Google

Colab)

Connectez votre notebook Google Colab à un jeton d'exécution avec GPU (Modifier > Paramètres du notebook > Accélérateur matériel > T4 GPU).
Écrivez et exécutez un script dans Colab pour valider l'environnement matériel et logiciel de développement :

```
# 1. Vérification du système d'exploitation et de
l'architecture matérielle
!uname -a
!cat /proc/cpuinfo | grep "model name" | head -n 1

# 2. Vérification de la présence du pilote graphique NVIDIA
et de la version CUDA
!nvidia-smi

# 3. Code Python : Vérification de l'accessibilité du GPU par
le Framework
import torch
print("PyTorch Version :", torch.__version__)
print("Est-ce que CUDA (GPU) est disponible ?",
torch.cuda.is_available())
if torch.cuda.is_available():
    print("Nom du GPU détecté :",
torch.cuda.get_device_name(0))
```

Travail demandé : Exécutez ce bloc. Inspectez la sortie textuelle de la commande nvidia-smi. Quelle est la version exacte du driver NVIDIA installé sur votre instance Colab ? Quelle est la quantité totale de mémoire vidéo (VRAM) disponible sur ce GPU ?

Partie 3 : Virtualisation - Conception d'un Dockerfile pour l'IA

Pour s'assurer que le code écrit dans Colab s'exécutera de la même manière sur les serveurs de production d'AWS ou de Google Cloud, vous devez créer une configuration de conteneurisation Docker.

Travail demandé : Complétez les lignes manquantes (représentées par des ???) de

ce modèle de Dockerfile dans votre rendu de projet pour installer correctement Python, PyTorch et copier un script nommé train.py.

```
# Étape 1 : Choisir une image de base officielle NVIDIA contenant CUDA et Ubuntu
FROM nvidia/cuda:12.1.1-runtime-ubuntu22.04

# Étape 2 : Mettre à jour le gestionnaire de paquets Linux et installer Python
RUN apt-get update && apt-get install -y \
    python3-pip \
    python3-dev \
    && rm -rf /var/lib/apt/lists/*

# Étape 3 : Installer le framework d'apprentissage machine (PyTorch et Pandas)
RUN pip3 install ??? ???

# Étape 4 : Définir le répertoire de travail à l'intérieur du conteneur
WORKDIR /app

# Étape 5 : Copier les fichiers de code locaux dans le conteneur
COPY train.py /app/

# Étape 6 : Définir la commande par défaut pour lancer l'exécution du script
CMD ["python3", "???"]
```

Partie 4 : Programmation d'un Agent de Monitoring et Watchdog (Python)

Vous devez maintenant concevoir la brique logicielle de contrôle d'infrastructure (Module 5). Vous allez écrire un script de surveillance active en Python qui simule la réception de métriques toutes les secondes et déclenche des contre-mesures (alertes ou redémarrage simulé) si des signes de défaillance surviennent.

Travail demandé : Copiez, complétez et exécutez le script ci-dessous dans votre notebook Google Colab.

```
import time
import random

def watchdog_logger(temps, temp_gpu, errors_ecc):
    print(f"[LOG - Sec {temps}] Température GPU: {temp_gpu}°C | Erreurs ECC: {errors_ecc}")

    # RÈGLE 1 : Si la température dépasse 82°C, lever une alerte de "Thermal Throttling"
    if temp_gpu > 82:
        print("  --> [ALERT SYSTEM] Vitesse d'horloge bridée automatiquement (Thermal Throttling) !")

    # RÈGLE 2 : Si le nombre d'erreurs ECC dépasse 10, simuler une vidange ("Drain") du serveur
    # COMPLÉTER ICI : Écrire la condition pour les erreurs ECC
    if ???:
        print("  --> [CRITICAL ERROR] Trop d'erreurs ECC ! Isolation du nœud et basculement du calcul.")
        return False # Arrêt du système de surveillance pour maintenance

    return True

# Simulation d'un entraînement d'IA sur 10 secondes avec variations de télémétrie
continuer_execution = True
for seconde in range(1, 11):
    if not continuer_execution:
        break

    # Génération de métriques aléatoires réalistes
    temp_simulee = random.randint(70, 86)
    # Simulation d'une augmentation progressive d'erreurs
```

```
mémoires défectueuses
    ecc_simulees = 0 if seconde < 6 else (seconde - 5) * 4

    # Appel de notre chien de garde (Watchdog)
    continuer_execution = watchdog_logger(seconde,
temp_simulee, ecc_simulees)
    time.sleep(1)
```

3. Critères d'Évaluation et Modalités de Remise

Le projet doit être remis avant le début de la prochaine séance de cours sous la forme d'un document texte ou d'un lien partagé vers votre notebook Google Colab contenant l'ensemble de vos réponses et codes exécutés.

- **Rigueur de l'analyse matérielle (Partie 1) :** 25 points
- **Exécution et justesse des commandes systèmes (Partie 2) :** 25 points
- **Syntaxe et logique du Dockerfile (Partie 3) :** 25 points
- **Fonctionnement de la boucle d'alerte du Watchdog (Partie 4) :** 25 points

Bon travail à tous ! Prenez le temps d'analyser les retours d'exécution de vos scripts de monitoring pour bien comprendre les impacts sur la chaîne matérielle.